

TD

On définit une liste chaînée comme suit :

```
struct node
{
    int data;
    struct node *p_next;
    struct node *p_prev;
};
```

Pour représenter notre liste chaînée à proprement parler, nous utiliserons une deuxième liste que voici :

```
typedef struct dlist
{
    size_t length;
    struct node *p_tail;
    struct node *p_head;
} Dlist;
```

```
Dlist *list = NULL; /* Déclaration d'une liste vide */
```

Afin de bien visualiser nos pointeurs, nous utiliserons le préfixe p_ pour toutes nos variables pointeurs.

1. Créez une fonction vous permettant d'allouer de l'espace mémoire pour votre liste chaînée. La fonction retournera la liste chaînée nouvellement créée.

```
Dlist *dlist_new(void)
{
    Dlist *p_new = malloc(sizeof *p_new);
    if (p_new != NULL)
    {
        p_new->length = 0;
        p_new->p_head = NULL;
        p_new->p_tail = NULL;
    }
    return p_new;
}
```

2. Ecrire une fonction qui ajoute un élément en fin de liste.

```
Dlist *dlist_append(Dlist *p_list, int data)
{
    if (p_list != NULL) /* On vérifie si notre liste a été allouée */
    {
        struct node *p_new = malloc(sizeof *p_new); /* Création d'un
nouveau node */
        if (p_new != NULL) /* On vérifie si le malloc n'a pas échoué */
        {
            p_new->data = data; /* On 'enregistre' notre donnée */
            p_new->p_next = NULL; /* On fait pointer p_next vers NULL */
            if (p_list->p_tail == NULL) /* Cas où notre liste est vide
(pointeur vers fin de liste à NULL) */
            {
```

```

        p_new->p_prev = NULL; /* On fait pointer p_prev vers NULL
*/
        p_list->p_head = p_new; /* On fait pointer la tête de liste
vers le nouvel élément */
        p_list->p_tail = p_new; /* On fait pointer la fin de liste
vers le nouvel élément */
    }
    else /* Cas où des éléments sont déjà présents dans notre liste
*/
    {
        p_list->p_tail->p_next = p_new; /* On relie le dernier
élément de la liste vers notre nouvel élément (début du chaînage) */
        p_new->p_prev = p_list->p_tail; /* On fait pointer p_prev
vers le dernier élément de la liste */
        p_list->p_tail = p_new; /* On fait pointer la fin de liste
vers notre nouvel élément (fin du chaînage: 3 étapes) */
    }
    p_list->length++; /* Incrémentement de la taille de la liste */
}
return p_list; /* on retourne notre nouvelle liste */
}

```

3. Ecrire une fonction qui ajoute un élément en début de liste :

```

Dlist *dlist_prepend(Dlist *p_list, int data)
{
    if (p_list != NULL)
    {
        struct node *p_new = malloc(sizeof *p_new);
        if (p_new != NULL)
        {
            p_new->data = data;
            p_new->p_prev = NULL;
            if (p_list->p_tail == NULL)
            {
                p_new->p_next = NULL;
                p_list->p_head = p_new;
                p_list->p_tail = p_new;
            }
            else
            {
                p_list->p_head->p_prev = p_new;
                p_new->p_next = p_list->p_head;
                p_list->p_head = p_new;
            }
            p_list->length++;
        }
    }
    return p_list;
}

```

4. On désire maintenant ajouter un élément n'importe où dans notre liste. Créez une fonction pour ceci.

```

Dlist *dlist_insert(Dlist *p_list, int data, int position)
{
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_head;
        int i = 1;
        while (p_temp != NULL && i <= position)
        {

```

```

        if (position == i)
        {
            if (p_temp->p_next == NULL)
            {
                p_list = dlist_append(p_list, data);
            }
            else if (p_temp->p_prev == NULL)
            {
                p_list = dlist_prepend(p_list, data);
            }
            else
            {
                struct node *p_new = malloc(sizeof *p_new);
                if (p_new != NULL)
                {
                    p_new->data = data;
                    p_temp->p_next->p_prev = p_new;
                    p_temp->p_prev->p_next = p_new;
                    p_new->p_prev = p_temp->p_prev;
                    p_new->p_next = p_temp;
                    p_list->length++;
                }
            }
        }
        else
        {
            p_temp = p_temp->p_next;
        }
        i++;
    }
    return p_list;
}

```

5. Ecrire la fonction qui libère la liste.

```

void dlist_delete(Dlist **p_list)
{
    if (*p_list != NULL)
    {
        struct node *p_temp = (*p_list)->p_head;
        while (p_temp != NULL)
        {
            struct node *p_del = p_temp;
            p_temp = p_temp->p_next;
            free(p_del);
        }
        free(*p_list), *p_list = NULL;
    }
}

```

6. Ecrivez une fonction dlist_display prenant en paramètre une liste et affichant tous les éléments de la liste séparés par des '>'. Lorsque la fin de liste sera atteinte, NULL sera affiché. Exemple :

4 > 8 > 15 > 16 > 23 > 42 > NULL

```

void dlist_display(Dlist *p_list)
{
    if (p_list != NULL)
    {

```

```

    struct node *p_temp = p_list->p_head;
    while (p_temp != NULL)
    {
        printf("%d -> ", p_temp->data);
        fflush(stdout);
        p_temp = p_temp->p_next;
    }
    printf("NULL\n");
}

```

7. Ecrire une fonction qui supprime un élément suivant sa valeur.

```

Dlist *dlist_remove(Dlist *p_list, int data)
{
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_head;
        int found = 0;
        while (p_temp != NULL && !found)
        {
            if (p_temp->data == data)
            {
                if (p_temp->p_next == NULL)
                {
                    p_list->p_tail = p_temp->p_prev;
                    p_list->p_tail->p_next = NULL;
                }
                else if (p_temp->p_prev == NULL)
                {
                    p_list->p_head = p_temp->p_next;
                    p_list->p_head->p_prev = NULL;
                }
                else
                {
                    p_temp->p_next->p_prev = p_temp->p_prev;
                    p_temp->p_prev->p_next = p_temp->p_next;
                }
                free(p_temp);
                p_list->length--;
                found = 1;
            }
            else
            {
                p_temp = p_temp->p_next;
            }
        }
    }
    return p_list;
}

```

8. Créez une fonction permettant de supprimer le n-ème élément d'une liste doublement chaînée.

```

Dlist *dlist_remove_id(Dlist *p_list, int position)
{
    if (p_list != NULL)
    {
        struct node *p_temp = p_list->p_head;
        int i = 1;
        while (p_temp != NULL && i <= position)
        {
            if (position == i)

```

```
    {
        if (p_temp->p_next == NULL)
        {
            p_list->p_tail = p_temp->p_prev;
            p_list->p_tail->p_next = NULL;
        }
        else if (p_temp->p_prev == NULL)
        {
            p_list->p_head = p_temp->p_next;
            p_list->p_head->p_prev = NULL;
        }
        else
        {
            p_temp->p_next->p_prev = p_temp->p_prev;
            p_temp->p_prev->p_next = p_temp->p_next;
        }
        free(p_temp);
        p_list->length--;
    }
    else
    {
        p_temp = p_temp->p_next;
    }
    i++;
}
return p_list;
}
```